

Antigravity CLI Setup for ROS2

https://gemini.google.com/app/49ff3d61663b725e

User prompt: hey Gem, *настроенное* *восхитительное*, *настроние* *спасми* *мур* but lets start with a trivial one. setup Antigravity CLI for ROS2 demo project with Gazebo/Three.js driven simulation. I need your help in maping Claude plugins to the Antigravity. I have the following plugins in Claude `rust-analyzer-lsp Plugin · claude-plugins-official · ✓ enabled · claude-code-setup Plugin · claude-plugins-official · ✓ enabled` `claude-mem Plugin · thedotmack · ✓ enabled` `└ mcp-search MCP · ✓ connected` `code-review Plugin · claude-plugins-official · ✓ enabled` `code-simplifier Plugin · claude-plugins-official · ✓ enabled` `commit-commands Plugin · claude-plugins-official · ✓ enabled` `context7 Plugin · claude-plugins-official · ✓ enabled` `└ context7 MCP · ✓ connected` `feature-dev Plugin · claude-plugins-official · ✓ enabled` `figma Plugin · claude-plugins-official · ✓ enabled` `github Plugin · claude-plugins-official · ✓ enabled` `headroom Plugin · headroom-marketplace · ✓ enabled` `mattpocock-skills Plugin · claude-plugins-official · ✓ enabled` `pr-review-toolkit Plugin · claude-plugins-official · ✓ enabled` `ralph-loop Plugin · claude-plugins-official · ✓ enabled` `skill-creator Plugin · claude-plugins-official · ✓ enabled` `typescript-lsp Plugin · claude-plugins-official · ✓ enabled` and i want to ready to ROS2 Rust/Python/Preact/Typescript/Three.js development exclude that would be needed (eg figma), add skills, tools, plugins that will help list all plugins. tools, skills, and maybe even hooks i will need and explain how to set them up. I need caching plugin mapped to claude-mem, i need headroom analogy too.

Response: Antigravity CLI organizes agents through the `.agents/` workspace structure, lifecycle hooks (`hooks.json`), declarative skills (`SKILL.md`), and standard Model Context Protocol (MCP) servers.

Claude Plugins to Antigravity CLI Mapping

Claude Plugin / MCP	Antigravity CLI Equivalent	Implementation & Setup
claude-mem (<code>mcp-search</code>)	claude-mem (Antigravity mode) + <code>/learn</code>	Run <code>npx claude-mem install --ide antigravity</code> . It registers MCP tools in <code>mcp_config.json</code> and lifecycle hooks in <code>~/.gemini/settings.json</code> . Use <code>/learn</code> for explicit distillations.
headroom	PostToolUse output filter hook / Headroom Proxy	Pipe <code>run_command</code> outputs through an output trimmer (<code>rtk</code>) via <code>.agents/hooks.json</code> , or route agy through Headroom's local HTTP proxy.
rust-analyzer-lsp	<code>cargo</code> validation hooks + LSP MCP	Automated via a <code>PostToolUse</code> hook running <code>cargo check/clippy</code> , or an LSP-over-MCP server in <code>mcp_config.json</code> .
typescript-lsp	<code>tsc</code> hook + TypeScript LSP MCP	Add a <code>PostToolUse</code> lint hook running <code>tsc --noEmit</code> , plus <code>@modelcontextprotocol/server-filesystem</code> if needed.
context7 (<code>context7 MCP</code>)	context7 MCP	1:1 mapping. Declare Context7 under <code>.agents/mcp_config.json</code> .
feature-dev	<code>Native /plan</code> & <code>/grill-me</code>	Built-in CLI commands that scaffold reviewable implementation plans and conduct scoping interviews.
code-review / pr-review-toolkit	<code>Native /diff</code> + <code>.agents/skills/review</code>	Use the native diff viewer (<code>/diff</code>) paired with a custom <code>review/SKILL.md</code> .
code-simplifier	<code>Refactoring Skill</code> / <code>Rule</code>	Place code simplification guidelines in <code>.agents/rules/refactor.md</code> .
commit-commands	<code>Git CLI</code> / <code>Git Skill</code>	Native bash execution via <code>run_command</code> or workflow conventions defined in <code>.agents/rules/git.md</code> .
github	<code>GitHub MCP Server</code>	Run <code>@modelcontextprotocol/server-github</code> in <code>mcp_config.json</code> .
mattpocock-skills	<code>.agents/skills/mattpocock/</code>	Copy existing <code>SKILL.md</code> files directly; Antigravity uses the standard Agent Skills schema.

Claude Plugin / MCP	Antigravity CLI Equivalent	Implementation & Setup
ralph-loop	Native /boost & /goal	Use /boost for deep multi-agent verification and /goal for uninterrupted autonomous runs until all tests pass.
skill-creator	Native /learn & manual scaffolding	/learn auto-generates reusable skills from recent terminal turns; or place custom directories in .agents/skills/.
claude-code-setup	AGENTS.md + .agents/ structure	Project instructions live in AGENTS.md (or GEMINI.md) at root.
figma	Excluded	Omitted as requested.

Caching and the Headroom Analogy

1. Claude-Mem Mapping claude-mem includes native Antigravity CLI support:

```
npx claude-mem install --ide antigravity
```

This inspects your system, registers 8 lifecycle hooks (including SessionStart, BeforeAgent, and AfterTool), configures the worker service, and mounts the mcp-search server in .agents/mcp_config.json (or ~/.gemini/config/mcp_config.json).

2. Headroom Analogy in Antigravity Headroom functions as a proxy that trims verbose shell outputs (rtk), compacts markdown, and protects prompt cache headroom. In Antigravity CLI, you replicate this via two mechanisms:

- Prompt Caching & Context:** Antigravity runs with large context horizons and native prompt prefix caching. Use /context inside agy to inspect live context load.
- Command Output Trimming Hook:** In robotics workspaces, commands like colcon build or gz topic -e dump thousands of log lines that bloat agent memory. You handle this with a PostToolUse hook in .agents/hooks.json that truncates massive terminal outputs before the agent ingests them.

Workspace Configuration for ROS2, Gazebo, and Three.js

Initialize your project with the following workspace layout:

```
my_ros2_sim/
├── .agents/
│   ├── mcp_config.json
│   ├── hooks.json
│   ├── rules/
│   │   ├── ros2-conventions.md
│   │   └── threejs-sim.md
│   └── skills/
│       ├── ros2-rust-rclrs/
│       │   └── SKILL.md
│       └── urdf-threejs-sync/
│           └── SKILL.md
├── AGENTS.md
├── src/                # ROS2 packages (Rust & Python)
├── sim/                # Gazebo worlds & SDF/URDF assets
└── web/                # Preact + Three.js visualizer
```

1. MCP Configuration (.agents/mcp_config.json)

Connects GitHub, Context7, and a ROS2 CLI inspection bridge:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "YOUR_TOKEN"
      }
    },
    "context7": {
      "command": "npx",
      "args": ["-y", "context7-mcp"]
    },
    "claude-mem": {
      "command": "claude-mem-mcp"
    }
  }
}
```

2. Lifecycle Hooks (.agents/hooks.json)

Configures build verification, code quality, and command log compression:

```
{
  "enabled": true,
  "PostToolUse": [
    {
      "matcher": "run_command",
      "type": "command",
      "command": "bash -c 'if [ -f .agents/scripts/trim_logs.sh ]; then .agents/scripts/trim_logs.sh; fi'",
      "timeout": 10
    },
    {
      "matcher": "write_to_file|replace_file_content",
      "type": "command",
      "command": "bash -c 'if git diff --name-only | grep -E '\\\\.rs$\\\"; then cargo clippy --workspace --all-targets --message-format=short'",
      "timeout": 45
    },
    {
      "matcher": "write_to_file|replace_file_content",
      "type": "command",
      "command": "bash -c 'if git diff --name-only | grep -E '\\\\.ts|tsx$\\\"; then npm run --prefix web typecheck; fi'",
      "timeout": 30
    }
  ]
}
```

```
],
"PreToolUse": [
  {
    "matcher": "run_command",
    "type": "command",
    "command": "python3 -c 'import sys, json; cmd = sys.argv[1]; sys.exit(1 if any(b in cmd for b in [\"rm -rf /\", \"ros2 node kill -a\"]) else 0)'",
    "timeout": 5
  }
]
}
```

Skills & Rules to Add for This Stack

Create these targeted instructions in `.agents/rules/` and `.agents/skills/`:

- **Coordinate Frame Mapping (`.agents/rules/threejs-sim.md`):** Instructs the agent on coordinate disparities between ROS2/Gazebo (REP 103: *X*-forward, *Y*-left, *Z*-up) and Three.js (*X*-right, *Y*-up, *Z*-backward) to avoid inverted quaternion math when translating transforms over WebSockets (rosbridge_suite).
- **ROS2 Rust Integration (`.agents/skills/ros2-rust-rclrs/SKILL.md`):** Defines rules for rclrs client library patterns, executor setups, safe topic subscribers, and ensuring ament_cargo or colcon-cargo builds cleanly alongside Python nodes.
- **Gazebo Simulation Bridge (`.agents/skills/urdf-threejs-sync/SKILL.md`):** Provides patterns for loading URDF/SDF models in Three.js via urdf-loader, syncing link poses at 60 FPS using shared array buffers or binary WebSocket frames, and handling joint limits accurately.

Sources

- 1. [Antigravity CLI Setup - Claude-Mem](#)
- 2. [Headroom Claude Code Plugin & Skill: What It Actually Is](#)
- 3. [Google Antigravity CLI](#)

User prompt: lets delete claude-mem and add free analogy. i beleive, agentmemory will fit but feel free to suggest

Response: Replacing c laude -mem with a completely free, local alternative comes down to choosing between **knowledge graph memory** or **vector/document memory**.

Recommended Free Alternatives

Solution	Architecture	Storage Backend	Best Used For
@modelcontextprotocol/server-memory (Recommended)	Native MCP Knowledge Graph	Local JSON file	Tracking entities, relations, architecture decisions, and conventions across turns with zero setup.
agentmemory	Vector & Document Memory	Local ChromaDB / SQLite	Semantic search across historical code snippets, error logs, and multi-turn summaries.
mcp-server-sqlite	Relational / Structured	Local SQLite database	Explicit tabular state tracking, run logs, and simulation metrics.

Option 1: Official MCP Memory (Zero Setup, 100% Local)

The simplest drop-in replacement is the official reference MCP memory server. It builds a persistent knowledge graph locally on your disk, requires zero API keys, and needs no running background daemon.

Remove `claude-mem` and update `.agents/mcp_config.json`:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "YOUR_TOKEN"
      }
    },
    "context7": {
      "command": "npx",
      "args": ["-y", "context7-mcp"]
    },
    "memory": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-memory"]
    }
  }
}
```

The agent will automatically gain tools to create, read, and search entities and relations (`create_entities`, `search_nodes`, `read_graph`) stored locally at `~/.cache/modelcontextprotocol/` or project-relative depending on your OS.

Option 2: agentmemory (Vector & Semantic Memory)

If you specifically want embedding-backed semantic search (closer to how `claude-mem` uses embeddings for context retrieval), `agentmemory` works well because it defaults to a local ChromaDB instance or SQLite store without paid API requirements.

Because `agentmemory` is a Python client library rather than a standalone MCP binary, you expose it to Antigravity CLI in one of two ways:

A. As a Skill via Custom CLI Script

1. Install `agentmemory` inside your ROS2/Python virtual environment:

```
pip install agentmemory
```

2. Create `.agents/scripts/mem.py`:

```
#!/usr/bin/env python3
import sys
from agentmemory import create_memory, search_memories
```

```

action = sys.argv[1] if len(sys.argv) > 1 else "search"
category = sys.argv[2] if len(sys.argv) > 2 else "general"

```

```

if action == "save":
    text = sys.argv[3]
    create_memory(category, text)
    print(f"Stored memory in [{category}]")
elif action == "search":
    query = sys.argv[3] if len(sys.argv) > 3 else ""
    results = search_memories(category, query, count=5)
    for r in results:
        print(f"- {r['document']}")

```

3. Add a dedicated skill in `.agents/skills/memory/SKILL.md` instructing the agent to run `.agents/scripts/mem.py search <category> <query>` on session start or when looking up past architectural decisions.

B. As an MCP Python Server

If you prefer tools in the agent's schema instead of CLI commands, wrap `agentmemory` with `FastMCP`:

```

# .agents/servers/agentmemory_mcp.py
from mcp.server.fastmcp import FastMCP
from agentmemory import create_memory, search_memories

mcp = FastMCP("agentmemory")

@mcp.tool()
def save_memory(category: str, text: str) -> str:
    """Save persistent context, code decision, or convention."""
    create_memory(category, text)
    return f"Saved memory under category: {category}"

@mcp.tool()
def query_memory(category: str, query: str, count: int = 5) -> str:
    """Search stored memories by semantic similarity."""
    results = search_memories(category, query, count=count)
    return "\n".join([r["document"] for r in results]) if results else "No memories found."

if __name__ == "__main__":
    mcp.run()

```

Mount it in `.agents/mcp_config.json`:

```

"agentmemory": {
    "command": "python3",
    "args": [".agents/servers/agentmemory_mcp.py"]
}

```

Cleaning Up claude-mem Artifacts

To prevent orphan background workers and leftover hooks:

1. Stop any background claude-mem daemon:

```
pkill -f claude-mem || true
```
2. Check ~/.gemini/settings.json or your local .agents/hooks.json and remove any SessionStart / BeforeAgent hooks invoking claude-mem.
3. Delete .agents/mcp_config.json entries pointing to claude-mem-mcp or mcp-search.

User prompt: *is agentmemory lives aside @modelcontextprotocol/server-memory well? will it be a reasonable setup or redundant?*

Response: Running both side-by-side is **architecturally complementary but practically redundant** for most local CLI agent workflows. They solve memory from two fundamentally different angles, but having both often causes tool-selection friction unless you enforce a strict division of labor.

Dimension	@modelcontextprotocol/server-memory	agentmemory
Mechanism	Knowledge Graph (Entities & Relations)	Vector Store (ChromaDB Embeddings / Documents)
Lookup Style	Deterministic / Relational (Node A -> publishes -> Topic B)	Semantic Similarity ("find past Gazebo collision bugs")
Best For	Static architecture, coordinate frames, port configs, schema conventions	Freeform session notes, terminal error history, past debugging rationale
Weakness	Poor at fuzzy or unstructured recall	Struggles with exact relational hierarchies and strict facts
Operational Cost	Lightweight JSON, zero extra daemons	Runs local vector embedding pipelines (adds Python/Chroma overhead)

Where the Setup Creates Friction

1. **Tool Overlap and Token Bloat:** When an agent has both create_entities and save_memory, it often defaults to updating both, doubling memory-tool invocations and consuming context window tokens on redundant writes.
2. **Context Drift:** If you change a ROS2 topic name or a Three.js Euler convention, the Knowledge Graph can update that single entity node cleanly. The vector database will retain old, semantically similar chunks with the outdated topic, leading to conflicting retrieved context.
3. **Double Retrieval Latency:** On ambiguous queries, the agent will query both tools sequentially before taking action, noticeably slowing down response turnaround in Antigravity.

When Keeping Both Makes Sense

Running both is reasonable only if you enforce a **strict separation of responsibilities** in .agents/rules/memory-policy.md:

- **Knowledge Graph (@modelcontextprotocol/server-memory):** Reserve exclusively for concrete technical facts, ROS2 node architecture, QoS profiles, REP-103 frame conversions, and package dependencies.
- **Vector Store (agentmemory):** Reserve exclusively for unstructured logs, bug post-mortems, Gazebo rendering error workarounds, and cross-session task status.

Recommendation for Your Stack

For a **ROS2 + Gazebo + Three.js** environment, precision matters significantly more than fuzzy recall:

- ROS2 development hinges on rigid, exact details: topic names, message field types, QoS depth, coordinate handedness, and joint names in URDFs.
- Semantic search often pulls "close enough" snippets that introduce subtle bugs (e.g., mixing up `geometry_msgs/msg/TransformStamped` with a custom pose topic).

Start solely with `@modelcontextprotocol/server-memory`. It is zero-maintenance, runs as a native lightweight node, and natively maps hierarchical robotic architectures without hallucinated semantic drift. Only introduce `agentmemory` if you find the knowledge graph failing to capture long, conversational problem-solving histories.

User prompt: *I like this harness file structure. translate it from Claude to Antigravity*

Response: Claude Code to Antigravity CLI Structural Mapping

Claude Code Component	Antigravity CLI Equivalent	Role & Execution Behavior
CLAUDE.md	AGENTS.md (or GEMINI.md)	Root instructions loaded at session start. Defines project overview, architecture, ROS2/Three.js tech stack, and primary build commands.
CLAUDE.local.md	AGENTS.local.md	Gitignored personal environment overrides (local simulation paths, local robot hardware URIs).
.mcp.json	.agents/mcp_config.json	Declares MCP servers (Context7, GitHub, Memory graph, local robotics bridges) shared via Git.
.claude/settings.json	.agents/settings.json	Project-level settings: tool whitelists, auto-approve policies, model parameters, and default context budgets.
.claude/settings.local.json	.agents/settings.local.json	Gitignored local developer settings (e.g., custom local binary paths for Gazebo).
.claude/rules/	.agents/rules/	Scoped guidelines loaded based on topic or file glob matching (e.g., coordinate frames, Rust formatting, Three.js shaders).
.claude/commands/	.agents/commands/	Custom slash command workflows (<code>/test-sim</code> , <code>/build-rust</code>) wrapping predefined prompts and tool sequences.
.claude/skills/	.agents/skills/	Agent Skills standard. Folders containing SKILL.md loaded on-demand when relevant task triggers occur.
.claude/agents/	.agents/agents/	Sub-agent role definitions with isolated context windows, specialized system prompts, and restricted toolsets.
.claude/hooks/	.agents/hooks.json + .agents/hooks/	Event-driven triggers (<code>PreToolUse</code> , <code>PostToolUse</code> , <code>SessionStart</code>) orchestrating validation scripts and log trimmers.

Antigravity Directory Layout

```

my_ros2_project/
├── AGENTS.md
├── AGENTS.local.md                # (gitignored)
├── .agents/
│   ├── mcp_config.json           # MCP server declarations
│   ├── settings.json            # Project-level tool permissions & runtime defaults
│   ├── settings.local.json      # (gitignored)
│   ├── hooks.json               # Lifecycle event binding configurations
│   ├── hooks/                   # Executable hook scripts
│   │   ├── trim_logs.sh         # Headroom-style log truncation
│   │   └── validate_bash.sh     # Unsafe command blocker
│   ├── rules/                   # Scoped context rules
│   │   ├── code-style.md
│   │   ├── ros2-conventions.md
│   │   └── threejs-rep103.md
│   ├── commands/               # Custom workflow slash commands
│   │   ├── review.md           # /review
│   │   └── sim-build.md        # /sim-build
│   ├── skills/                 # Context-aware triggered capabilities
│   │   ├── ros2-rust-rclrs/
│   │   │   ├── SKILL.md
│   │   │   └── cargo-config.md
│   │   ├── urdf-sync/
│   │   │   └── SKILL.md
│   └── agents/                 # Specialized sub-agent personas
│       ├── code-reviewer.md
│       └── sim-auditor.md

```

Component Configuration Breakdown

1. Root Entrypoint (AGENTS.md) Replaces CLAUDE.md. Antigravity ingests this at session launch:

```

# Project Overview: ROS2 + Three.js Gazebo Visualizer

## Stack
- Backend: ROS2 (Iron/Humble), Rust (`rclrs`), Python (`rclpy`)
- Frontend: Preact, TypeScript, Three.js (`@react-three/fiber` equivalent)
- Sim Bridge: `rosbridge_suite`, Gazebo Fortress/Garden

## Commands
- Build Workspace: `colcon build --symlink-install`
- Run Web: `npm --prefix web run dev`

```

2. Scoped Rules (.agents/rules/threejs-rep103.md) Unlike global instructions, rule files govern specific subsystems and can specify file matchers:

```

---
apply_to: "web/**/*.{ts,tsx,js}"
---

```

```
# REP-103 to Three.js Frame Alignment
- ROS2 uses standard REP-103 (X-forward, Y-left, Z-up).
- Three.js uses standard WebGL conventions (X-right, Y-up, Z-backward).
- Always route transform quaternion re-mappings through math utilities; never apply raw ROS orientations directly to Three.js Scene objects.
```

3. Custom Slash Commands (.agents/commands/sim-build.md) Maps custom invocations directly into the Antigravity shell execution environment:

```
---
description: "Builds ROS2 packages and checks TypeScript web bindings"
---
1. Run `colcon build --symlink-install --packages-select ros2_rust_bridge`
2. Run `npm --prefix web run typecheck`
3. If errors occur, diagnose without altering coordinate bridge signatures.
```

4. Sub-Agent Personas (.agents/agents/sim-auditor.md) Replaces Claude's sub-agents by defining specialized roles with isolated context:

```
---
name: "sim-auditor"
tools: ["read_file", "run_command"]
denied_tools: ["write_to_file", "replace_file_content"]
---
You are a simulation physics auditor. Your sole task is inspecting URDF/SDF inertial tensors, joint effort limits, and contact collision pro
```

5. Hooks Configuration (.agents/hooks.json) Replaces loose .claude/hooks/ bindings with structured event triggers:

```
{
  "PreToolUse": [
    {
      "matcher": "run_command",
      "command": ".agents/hooks/validate_bash.sh"
    }
  ],
  "PostToolUse": [
    {
      "matcher": "run_command",
      "command": ".agents/hooks/trim_logs.sh"
    }
  ]
}
```

User prompt: *what about graphify to replace headroom?*

Response: **Graphify cannot directly replace Headroom** because they target token bloat at opposite ends of the agent execution lifecycle. However, Graphify is a free, local alternative to claude-mem and heavy repository indexing.

Functional Comparison

Capability	Headroom	Graphify
Primary Target	Dynamic command outputs & tool logs	Static codebase architecture & file reading
How It Saves Tokens	Intercepts terminal stdout/stderr (e.g., <code>colcon build</code> , <code>gz sim</code>) via a local proxy, reversibly compressing verbose logs.	Compiles the project into an AST knowledge graph via Tree-sitter, allowing targeted sub-graph queries (<code>graphify query</code>) instead of whole-file grepping.
Underlying Mechanism	Local-first proxy tray app with output trimming tools (<code>rtk</code>)	Deterministic AST parsing across 35+ languages without vector stores or LLM costs
Licensing	Paid subscription app (\$3–\$4/mo)	100% free and open-source (<code>graphify</code> on PyPI)
Best Used As	Shell log suppressor	Repository memory and context navigation

Why Graphify Replaces Claude-Mem Instead

While Headroom compresses what your ROS2 build tools emit, Graphify prevents the agent from reading 40 different Rust, TypeScript, and launch files just to understand your package dependencies:

- **No Vector Drift:** Unlike vector-based memory (`mem0`, `agentmemory`), Graphify parses deterministic links (calls, imports, inheritance) using Tree-sitter.
- **Zero Inference Overhead:** Extracting code structure requires zero LLM tokens.
- **REP-103 & Cross-Language Tracing:** It traces references across Python nodes, Rust crates, and Preact/Three.js components into a single `graph.json`.

Setup in Antigravity CLI

Graphify has native integration for Antigravity CLI workspaces.

1. Install Graphify

```
uv tool install graphify
```

(Note: PyPI uses *graphify* with two y's).

2. Register the Antigravity Skill

From your ROS2 project root:

```
graphify antigravity install
```

(Alternatively, run `graphify install --platform agents`).

This generates an on-demand skill under `.agents/skills/graphify/` and instructs Antigravity in `AGENTS.md` to consult `graphify-out/GRAPH_REPORT.md` before executing heavy greps or reading large batches of source files.

3. Generate the Initial Graph

Trigger the graph compilation inside your terminal or through the agent:

```
/graphify .
```

This builds an AST representation of your ROS2 nodes, URDF meshes, and Three.js visualizer code into `.agents/` cache without calling external cloud APIs.

Replicating Headroom Without Paying for It

Because Graphify does not compress shell logs, pair it with a free, local `PostToolUse` hook in `.agents/hooks.json` to handle Headroom's output trimming job:

```
{
  "PostToolUse": [
    {
      "matcher": "run_command",
      "command": "bash -c 'python3 -c \"import sys; lines = sys.stdin.readlines(); print(\\\"\\\"\\\".join(lines[:25] + [f\\\"\\\"\\\"\\n... [{len(lines
    ]
  ]
}
```

This captures build output bloat from `colcon build` or Gazebo physics dumps while Graphify handles context navigation.

For a visual demonstration of how Graphify queries subgraphs to cut agent token consumption across larger repositories, check out this [Graphify Token Reduction Breakdown](#). This video demonstrates how graph-based querying prevents coding agents from saturating their context windows during cross-file code navigation.

http://googleusercontent.com/youtube_content/1

User prompt: back to "Skills & Rules to Add for This Stack" please write the corresponding skills

Response: These configuration files, rules, and skills provide the domain-specific logic for your ROS2, Gazebo, and Three.js stack within the `.agents/` directory structure.

1. Coordinate Frame Mapping Rule

Create this file at `.agents/rules/threejs-rep103.md` to prevent coordinate system bugs between ROS REP 103 and WebGL:

```
---
apply_to: "web/**/*.{ts,tsx,js}"
---
# REP 103 to Three.js Frame Alignment

## Coordinate Conventions
* **ROS2 / Gazebo (REP 103):** Right-handed.  $+X$  is forward,  $+Y$  is left,  $+Z$  is up.
* **Three.js / WebGL:** Right-handed.  $+X$  is right,  $+Y$  is up,  $+Z$  is backward (towards camera).

## Position Translation
Convert ROS Cartesian vectors  $(x_{\text{ros}}, y_{\text{ros}}, z_{\text{ros}})$  to Three.js coordinates  $(x_{\text{three}}, y_{\text{three}}, z_{\text{three}})$  using:
 $x_{\text{three}} = -y_{\text{ros}}$ 
 $y_{\text{three}} = z_{\text{ros}}$ 
 $z_{\text{three}} = -x_{\text{ros}}$ 

## Rotation and URDF Alignment Rules
* **Avoid manual quaternion component swizzling.** Raw component swaps (e.g., swapping  $x$  and  $z$ ) break quaternion normalization and chira
* **Scene Root Rotation:** When using `urdf-loader`, keep the loaded `URDFRobot` inside a parent `THREE.Group`. Apply an initial  $-90^\circ$ 
```

```
```typescript
robotGroup.rotation.x = -Math.PI / 2;
```

Inside this rotated frame, URDF link translations and joint axes resolve natively to ROS standard coordinates.

- **Euler Order:** If converting Euler rotations, ROS standard is extrinsic RPY ( $Z$ - $Y$ - $X$  yaw-pitch-roll), whereas Three.js defaults to intrinsic 'XYZ'. Explicitly set `euler.reorder('ZYX')` before applying angles.

---

### ### 2. ROS2 Rust (`rclrs`) Development Skill

Create this file at ``.agents/skills/ros2-rust-rclrs/SKILL.md`` to guide the agent when creating, editing, or debugging Rust nodes:

```
```markdown
---
name: "ros2-rust-rclrs"
description: "Authoring, compiling, and debugging ROS2 nodes in Rust using rclrs and ament_cargo."
triggers:
  - "src/**/*.rs"
  - "**/Cargo.toml"
  - "colcon-cargo"
---
```

ROS2 Rust Development Conventions

Build System Architecture

- * Packages must contain both a valid ``package.xml`` and a ``Cargo.toml``.
- * The ``package.xml`` must set `<build_type>ament_cargo</build_type>`.
- * Build commands must invoke ``colcon build --packages-select <pkg_name> --symlink-install``.

Project Structure Example

```
```text
my_rust_node/
├── Cargo.toml
├── package.xml
├── src/
│ └── main.rs
```

#### package.xml Template

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema-instance"?>
<package format="3">
 <name>rust_sim_bridge</name>
 <version>0.1.0</version>
 <description>Rust ROS2 node for Gazebo telemetry</description>
```

```

<maintainer email="dev@local.net">Developer</maintainer>
<license>Apache-2.0</license>

<buildtool_depend>ament_cargo</buildtool_depend>
<depend>rclrs</depend>
<depend>std_msgs</depend>
<depend>sensor_msgs</depend>

<export>
 <build_type>ament_cargo</build_type>
</export>
</package>

```

## Node Implementation Pattern (src/main.rs)

Always follow this initialization and spin pattern to guarantee deterministic teardown:

```

use std::sync::Arc;
use rclrs::{Context, Node, RclrsError, QOS_PROFILE_DEFAULT};
use sensor_msgs::msg::JointState;

fn main() -> Result<(), RclrsError> {
 let context = Context::new(std::env::args())?;
 let node = rclrs::create_node(&context, "rust_sim_bridge")?;

 // Create a publisher for joint states
 let publisher = node.create_publisher::<JointState>("/joint_states", QOS_PROFILE_DEFAULT)?;

 // Create a subscription with callback
 let _subscription = node.create_subscription::<JointState, _>(
 "/gazebo/joint_states",
 QOS_PROFILE_DEFAULT,
 move |msg: JointState| {
 // Forward or process state
 if let Err(e) = publisher.publish(&msg) {
 eprintln!("Failed to publish joint state: {:?}", e);
 }
 },
)?;

 // Single-threaded execution loop
 rclrs::spin(node)
}

```

## Quality Guardrails

- **No blocking inside callbacks:** Never perform disk I/O, heavy matrix factorizations, or network requests inside the subscription callback.

- **Clippy:** Every Rust edit must pass `cargo clippy --workspace --all-targets -D warnings`.

```

3. URDF & Three.js Simulation Bridge Skill

Create this file at `.agents/skills/urdf-threejs-sync/SKILL.md` for building the web visualizer, handling mesh resolution, and streaming joi

```markdown
---
name: "urdf-threejs-sync"
description: "Loading URDF/SDF assets, mapping ROS package meshes, and streaming 60 FPS joint updates via WebSocket in Three.js."
triggers:
  - "web/**/*.{ts,tsx}"
  - "sim/**/*.{urdf}"
  - "sim/**/*.{sdf}"
---

# URDF & Three.js Web Synchronization

## Architecture Overview
* Model Loading: Use `urdf-loader` alongside Three.js loaders (`STLLoader`, `GLTFLoader`).
* Transport: Connect via WebSocket to `rosbridge_server` (default port: `9090`).
* Render Decoupling: Decouple incoming network packets from WebGL draw calls using an in-memory joint cache.

## 1. URDF Loading & Mesh Path Resolution
Map `package://<pkg_name>` URIs to the local HTTP asset directory:

```typescript
import { LoadingManager } from 'three';
import URDFLoader, { URDFRobot } from 'urdf-loader';

export function createRobotLoader(assetBaseUrl: string): URDFLoader {
 const manager = new LoadingManager();
 const loader = new URDFLoader(manager);

 // Map ROS package names to static asset directories
 loader.packages = {
 my_robot_description: `${assetBaseUrl}/models/my_robot_description`,
 };

 return loader;
}

```

## 2. High-Performance State Ingestion (Preact / TypeScript)

Do not trigger React/Preact component re-renders on every incoming ROS packet. Maintain a non-reactive dictionary of joint angles and apply them inside the animation frame loop:

```

import { useEffect, useRef } from 'preact/hooks';
import type { URDFRobot } from 'urdf-loader';

export function useJointStream(robotRef: { current: URDFRobot | null }, wsUrl: string) {
 // Store joint positions without triggering re-renders
 const latestJoints = useRef<Record<string, number>>({});

 useEffect(() => {
 const ws = new WebSocket(wsUrl);

 ws.onopen = () => {
 // Subscribe to ROS2 joint_states via rosbridge
 ws.send(JSON.stringify({
 op: 'subscribe',
 topic: '/joint_states',
 type: 'sensor_msgs/msg/JointState'
 }));
 };

 ws.onmessage = (event) => {
 const data = JSON.parse(event.data);
 if (data.op === 'publish' && data.msg?.name && data.msg?.position) {
 const names: string[] = data.msg.name;
 const positions: number[] = data.msg.position;
 for (let i = 0; i < names.length; i++) {
 latestJoints.current[names[i]] = positions[i];
 }
 }
 };

 return () => ws.close();
 }, [wsUrl]);

 // Call this function inside requestAnimationFrame or useFrame()
 const syncJointsToMesh = () => {
 const robot = robotRef.current;
 if (!robot) return;

 for (const [name, angle] of Object.entries(latestJoints.current)) {
 if (robot.joints[name]) {
 robot.setJointValue(name, angle);
 }
 }
 };

 return { syncJointsToMesh };
}

```

## Implementation Rules

- **Joint Value Clamping:** `robot.setJointValue` strictly adheres to URDF `<limit lower="..." upper="...">` tags. If a joint fails to move, check for out-of-bounds radians in the ROS telemetry stream.
- **Continuous Joints:** For wheels or continuous joints (`type="continuous"`), ensure values do not overflow over long runs; normalize angles to  $[-\pi, \pi]$  if precision loss occurs.
- **Shadows & Colliders:** By default, invoke `loader.parseCollision = false` during runtime visualization to cut draw calls in half. Load collision hulls only when collision-debugging mode is explicitly enabled.

```

Activating the Rules and Skills

Add these new skills to `.agents/mcp_config.json` or your project settings so the agent indexes them:

1. Place the three files into:
 * `.agents/rules/threejs-rep103.md`
 * `.agents/skills/ros2-rust-rclrs/SKILL.md`
 * `.agents/skills/urdf-threejs-sync/SKILL.md`
2. Add reference anchors to `.AGENTS.md` under a ``` Coding Guidelines` header:
  ```markdown  
  ## Guidelines  
  - Coordinate frames and WebGL mapping: see `.agents/rules/threejs-rep103.md`.  
  - ROS2 Rust node design: see `.agents/skills/ros2-rust-rclrs/SKILL.md`.  
  - Three.js / Gazebo sync: see `.agents/skills/urdf-threejs-sync/SKILL.md`.
```

User prompt: `.agents/skills/ros2-rust-rclrs/SKILL.md` is badly rendered here in chat. regenerate

```
Response: ---  
name: "ros2-rust-rclrs"  
description: "Authoring, compiling, and debugging ROS2 nodes in Rust using rclrs and ament_cargo."  
triggers:  
  - "src/**/*.rs"  
  - "**/Cargo.toml"  
  - "colcon-cargo"  
---
```

ROS2 Rust Development Conventions

Build System Architecture

- * Every package must contain both a valid ``.package.xml`` and a ``.Cargo.toml``.
- * The ``.package.xml`` must declare `<build_type>ament_cargo</build_type>`.
- * Build invocations must target specific packages: ``.colcon build --packages-select <pkg_name> --symlink-install``.

```
## Project File Layout
```

```
``text
my_rust_node/
├─ Cargo.toml
├─ package.xml
├─ src/
│   └─ main.rs
...
```

```
## Configuration Files
```

```
### `package.xml`
```

```
``xml
<?xml version="1.0"?>
<?xml-model href="[http://download.ros.org/schema/package_format3.xsd](http://download.ros.org/schema/package_format3.xsd)" schematypens="http://download.ros.org/schema/package_format3.xsd" package_format="3">
  <name>rust_sim_bridge</name>
  <version>0.1.0</version>
  <description>Rust ROS2 node for Gazebo telemetry</description>
  <maintainer email="dev@local.net">Developer</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cargo</buildtool_depend>
  <depend>rclrs</depend>
  <depend>std_msgs</depend>
  <depend>sensor_msgs</depend>

  <export>
    <build_type>ament_cargo</build_type>
  </export>
</package>
``
```

```
### `Cargo.toml`
```

```
``toml
[package]
name = "rust_sim_bridge"
version = "0.1.0"
edition = "2021"
```

```
[dependencies]
rclrs = "*"
sensor_msgs = "*"
std_msgs = "*"
``
```

```
## Node Implementation Pattern (`src/main.rs`)
```

```

```rust
use std::sync::Arc;
use rclrs::{Context, Node, RclrsError, QOS_PROFILE_DEFAULT};
use sensor_msgs::msg::JointState;

fn main() -> Result<(), RclrsError> {
 // Initialize ROS2 execution context
 let context = Context::new(std::env::args())?;
 let node = rclrs::create_node(&context, "rust_sim_bridge")?;

 // Create publisher for downstream telemetry
 let publisher = node.create_publisher::<JointState>("/joint_states", QOS_PROFILE_DEFAULT)?;

 // Create subscriber forwarding Gazebo states
 let _subscription = node.create_subscription::<JointState, _>(
 "/gazebo/joint_states",
 QOS_PROFILE_DEFAULT,
 move |msg: JointState| {
 if let Err(e) = publisher.publish(&msg) {
 eprintln!("Failed to publish joint state: {:?}", e);
 }
 },
);

 // Spin executor loop until termination signal
 rclrs::spin(node)
}
```

## Quality Guardrails
* **Non-blocking Callbacks:** Subscription callbacks must execute quickly without performing disk I/O, heavy transformations, or blocking ne
* **Linter Compliance:** All Rust code changes must cleanly pass:
    ```bash
 cargo clippy --workspace --all-targets -D warnings
    ```

```

User prompt: Add these new skills to `.agents/mcp_config.json` or your project settings so the agent indexes them here is my current `mcp_config.json` send the updated one ```json {
 "mcpServers": {
 "github": {
 "command": "npx",
 "args": ["-y", "@modelcontextprotocol/server-github"],
 "env": {
 "GITHUB_PERSONAL_ACCESS_TOKEN": "YOUR_TOKEN"
 }
 },
 "context7": {
 "command": "npx",
 "args": ["-y", "context7-mcp"],
 "memory": {
 "command": "npx",
 "args": ["-y", "@modelcontextprotocol/server-memory"]
 }
 }
 }
} ```

Response: Skills and rules are **not** registered inside `mcp_config.json`. That file is reserved strictly for executable Model Context Protocol (MCP) server processes communicating over stdio.

Antigravity CLI auto-discovers skills natively from the filesystem when they reside in `.agents/skills/<name>/SKILL.md`.

Clean `.agents/mcp_config.json`

Your MCP configuration needs no extra entries for markdown skills (closing brace fixed from your snippet):

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "YOUR_TOKEN"
      }
    },
    "context7": {
      "command": "npx",
      "args": ["-y", "context7-mcp"]
    },
    "memory": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-memory"]
    }
  }
}
```

Where Skills & Rules Actually Get Indexed

1. Filesystem Placement (Automatic Discovery) Antigravity scans `.agents/skills/` on launch. As long as the directory layout matches standard Agent Skills schema, the skills are ready to trigger:

```
.agents/
├── rules/
│   └── threejs-rep103.md
└── skills/
    ├── ros2-rust-rclrs/
    │   └── SKILL.md
    └── urdf-threejs-sync/
        └── SKILL.md
```

2. Optional Explicit Registration (`.agents/settings.json`) If you enforce strict skill allowlists or permissions in your project settings, declare them in `.agents/settings.json`:

```
{
  "skills": {
    "enabled": [
      "ros2-rust-rclrs",
      "urdf-threejs-sync"
    ]
  },
  "rules": {
    "include": [
      ".agents/rules/*.md"
    ]
  }
}
```

```
}  
}
```

3. Root Anchor (AGENTS.md) To ensure the model proactively pulls the skills without waiting for trigger keywords:

Agent Directives

Domain Skills & Standards

- ****ROS2 & Rust (`rclrs`)**** Follow `.agents/skills/ros2-rust-rclrs/SKILL.md` for package layout and node design.
 - ****URDF & Three.js**** Follow `.agents/skills/urdf-threejs-sync/SKILL.md` for runtime mesh binding and WebSocket streaming.
 - ****Transform Math**** Strict adherence to REP 103 vs WebGL coordinates in `.agents/rules/threejs-rep103.md`.
-

User prompt: and overall rust dev support on the harness level

Response: To provide comprehensive Rust development support at the harness level, Antigravity CLI relies on four components: **event-driven feedback hooks** (auto-linting/formatting), **LSP navigation via MCP**, **reusable slash commands**, and **scoped compiler rules**.

1. Event-Driven Feedback Loop (.agents/hooks.json)

Instead of waiting for you to test code manually, configure hooks to immediately run `rustfmt` and `cargo clippy` whenever the agent modifies any `.rs` file.

Update `.agents/hooks.json`:

```
{  
  "PostToolUse": [  
    {  
      "matcher": "write_to_file|replace_file_content",  
      "command": ".agents/hooks/rust_feedback.sh"  
    }  
  ]  
}
```

Create `.agents/hooks/rust_feedback.sh` (make executable with `chmod +x`):

```
#!/usr/bin/env bash  
set -e  
  
# Extract changed Rust files  
CHANGED_RS=$(git status --porcelain | awk '{print $2}' | grep -E '\.rs$' || true)  
  
if [ -n "$CHANGED_RS" ]; then  
  # 1. Deterministic formatting  
  echo "$CHANGED_RS" | xargs rustfmt --edition 2021 2>/dev/null || true  
  
  # 2. Fast compiler & clippy check (short format to protect context window)  
  # ROS2 packages require ROS sourcing if rclrs bindings are linked  
  [ -f /opt/ros/humble/setup.bash ] && source /opt/ros/humble/setup.bash 2>/dev/null || true
```

```
cargo clippy --workspace --all-targets --message-format=short 2>&1 | head -n 30
fi
```

When this hook fails, Antigravity feeds compiler and borrow-checker diagnostics directly back to the agent in the same turn, triggering automatic self-correction before committing changes.

2. Rust LSP via Model Context Protocol (.agents/mcp_config.json)

To replace Claude's rust-analyzer-lsp plugin, register a local rust-analyzer MCP server. This gives the agent deep symbol resolution, type definition jumping, and workspace diagnostics without grepping.

Add rust-analyzer-mcp to .agents/mcp_config.json:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "YOUR_TOKEN"
      }
    },
    "context7": {
      "command": "npx",
      "args": ["-y", "context7-mcp"]
    },
    "memory": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-memory"]
    },
    "rust-analyzer": {
      "command": "npx",
      "args": ["-y", "rust-analyzer-mcp"]
    }
  }
}
```

(Prerequisite: ensure rust-analyzer is installed locally via rustup component add rust-analyzer).

3. Harness Slash Commands (.agents/commands/)

Create shortcut workflows in .agents/commands/ for targeted development cycles:

A. Fast Workspace Verification (.agents/commands/cargo-check.md)

```
---
description: "Executes targeted cargo check with R0S2 environment sourced"
---
```

1. Run ``cargo check --workspace --tests --all-targets``
2. If ROS2 client libraries fail to link, verify ``source /opt/ros/$ROS_DISTRO/setup.bash`` was run.
3. Report any unused variables or unhandled Result types.

B. Rust Test Diagnostics (.agents/commands/cargo-test.md)

```
---
description: "Runs cargo tests with formatted failure summaries"
---
1. Execute `cargo test --workspace -- --nocapture`
2. If assertions fail, identify the exact test location, expected vs actual values, and propose an inline fix.
```

4. Compiler & Language Rules (.agents/rules/rust.md)

Create .agents/rules/rust.md to prevent common LLM anti-patterns (e.g., abusing `.unwrap()`, improper error masking, or missing ROS2 types):

```
---
apply_to: "src/**/*.rs"
---
# Rust Development Standards

## Error Handling
* Never use `unwrap()` or `expect()` in production node logic or message callbacks.
* Use `?` operator bubbling up to `Result<(), RclrsError>` or typed domain errors.
* Provide explicit context using crates like `thiserror` for custom library components.

## Borrow Checker & Concurrency
* Prefer message passing or atomic primitives (`Arc<AtomicBool>`) over heavy `Arc<Mutex<T>>` hierarchies inside ROS subscriber callbacks.
* Avoid keeping `MutexGuard` across `await` points or long blocking calls.

## Memory & Performance
* Avoid allocating dynamic vectors or strings inside high-frequency simulation callbacks (`>50 Hz`).
* Reuse pre-allocated scratch buffers when parsing sensor packets (`LaserScan`, `PointCloud2`).
```

5. Environment Synchronization (AGENTS.md)

Because ROS2 Rust nodes (`rclrs`) bind directly to underlying C ROS libraries (`librcl`, `rmw`), Cargo must recognize ROS include directories. Ensure this directive is present in your root AGENTS.md:

```
## Rust & Cargo Execution
- Always prepend ROS2 environment variables before running raw cargo commands:
  `source /opt/ros/$ROS_DISTRO/setup.bash && cargo <command>`
- For root workspace builds containing mixed Rust/Python packages:
  `colcon build --symlink-install --packages-select <rust_package_name>`
```

User prompt: you are experienced senior rust developer. you use latest convention and utilise modern rust ecosystem tools. i want to highlight that specifically and make re-generate the task like a PRO. eg, rust-clippy maybe a good choice for linting if you dont know smt better. then, never use unwrap() is too harsh and probably will create more problems then benefits.

Response: A seasoned Rust setup avoids cargo-cult dogmas like "never use unwrap()" and replaces brittle CLI flags with modern workspace-level lint tables, fast test runners (cargo-nextest), and deterministic invariant validation.

1. Modern Workspace Configuration (Cargo.toml)

Modern Rust centralizes linting and compiler diagnostics in Cargo.toml via [workspace.lints]. This keeps your hooks and CLI commands short while enforcing granular, idiomatic checks across every crate in the workspace.

```
[workspace]
resolver = "2"
members = ["src/*"]
```

```
[workspace.package]
version = "0.1.0"
edition = "2021"
rust-version = "1.78"
license = "Apache-2.0"
```

```
[workspace.lints.rust]
unsafe_code = "deny"
missing_debug_implementations = "warn"
unreachable_pub = "warn"
```

```
[workspace.lints.clippy]
all = { level = "warn", priority = -1 }
pedantic = { level = "warn", priority = -1 }
nursery = { level = "warn", priority = -1 }
```

```
# Downgrade pedantic rules that generate low-value noise
module_name_repetitions = "allow"
must_use_candidate = "allow"
doc_markdown = "allow"
```

```
# Critical reliability & safety checks
undocumented_unsafe_blocks = "deny"
unwrap_used = "warn"           # Warn, don't deny: surface for review without breaking prototypes
fallible_impl_from = "deny"
as_conversions = "warn"
clone_on_ref_ptr = "warn"     # Flags accidental deep clones vs Arc clones
```

2. Modern Error Handling and Invariant Rules (.agents/rules/rust.md)

Replace dogmatic anti-unwrap directives with production-grade invariant handling:

```

---
apply_to: "src/**/*.rs"
---
# Idiomatic Rust Architecture Guidelines

## Invariant Safety vs. Operational Errors
* **Operational Failures (Fallible Path):**
  * Network I/O, topic deserialization, QoS handshake mismatches, and file reads must return typed domain errors using `thiserror` (for lib)
  * Never silence errors with `.unwrap_or_default()` unless the default value is explicitly valid domain behavior.
* **Invariants & Panic Scenarios:**
  * `.expect("descriptive failure reason")` is completely valid when upholding an unrecoverable internal invariant (e.g., regex compilation
  * Lock poisoning: write `mutex.lock().expect("mutex poisoned by failing thread")`. Attempting to recover from a poisoned mutex in a real-t
* **Prototyping & Unit Tests:**
  * `.unwrap()` and `.expect()` are standard in tests, benchmarks, and binary CLI entrypoints.

## Real-Time Constraints & Allocations (ROS2 / Simulation)
* **Zero-Allocation Hot Paths:** Inside subscriber callbacks (>50 Hz), avoid heap allocations (`String`, `Vec::clone()`, `Box`).
* **Fixed Buffers:** Use `arrayvec::ArrayVec` or `smallvec::SmallVec` for fixed-capacity sensor vectors (e.g., IMU quaternions, transform ch
* **Concurrency:** Favor atomics (`std::sync::atomic`) or bounded lock-free channels (`crossbeam-channel`) over heavy `Arc<Mutex<T>>` struct

```

3. Event-Driven Developer Feedback Hook (.agents/hooks.json)

Configure a fast, non-blocking feedback hook that runs rustfmt, builds with cargo check, and executes tests using **cargo-nextest** (the modern standard for isolated, parallel test execution).

.agents/hooks.json

```

{
  "PostToolUse": [
    {
      "matcher": "write_to_file|replace_file_content",
      "command": ".agents/hooks/rust_feedback.sh"
    }
  ]
}

```

.agents/hooks/rust_feedback.sh

```

#!/usr/bin/env bash
set -eo pipefail

# Find modified Rust files
CHANGED_RS=$(git status --porcelain | awk '{print $2}' | grep -E '\.rs$' || true)

if [ -n "$CHANGED_RS" ]; then
  # Source ROS2 workspace environment if required for rclrs bindings
  if [ -z "$ROS_DISTRO" ] && [ -f /opt/ros/humble/setup.bash ]; then

```

```

    source /opt/ros/humble/setup.bash
fi

# 1. Format dirty files instantly
echo "$CHANGED_RS" | xargs rustfmt --edition 2021 2>/dev/null || true

# 2. Workspace diagnostics using Cargo.toml [workspace.lints]
# Using short-format output to keep agent context tight
cargo clippy --workspace --all-targets --message-format=short 2>&1 | head -n 35

# 3. Incremental unit test pass (only run unit tests, skip heavy integration/sim tests)
if command -v cargo-nextest >/dev/null 2>&1; then
    cargo nextest run --workspace --lib -q 2>&1 | tail -n 15 || true
else
    cargo test --lib --workspace -q 2>&1 | tail -n 15 || true
fi
fi

```

4. Advanced Cargo Tooling via Agent Slash Commands

Give the agent modern CLI workflows under `.agents/commands/`:

1. Modern Test Suite (`.agents/commands/test.md`)

```

---
description: "Run workspace tests via cargo-nextest with structured output"
---
1. Execute: `cargo nextest run --workspace --all-targets`
2. If any test fails, run the exact failing test under isolated output:
   `cargo nextest run -E 'test(test_name)' --no-capture`
3. Identify whether failure stems from coordinate conversion drift or state timeout.

```

2. Dependency Hygiene & Leak Check (`.agents/commands/audit.md`)

```

---
description: "Scan for unused dependencies, security advisories, and licence compliance"
---
1. Check for unused dependencies: `cargo machete`
2. Check security vulnerabilities and bans: `cargo deny check`
3. Report any circular dependencies or duplicate transitive crates.

```

5. rust-analyzer LSP Configuration (`.agents/mcp_config.json`)

Pass explicit build options to rust-analyzer so it understands your ROS2 target and workspace build targets without chewing through excessive RAM:

```

{
  "mcpServers": {
    "rust-analyzer": {

```

```

"command": "npx",
"args": ["-y", "rust-analyzer-mcp"],
"env": {
  "RA_LOG": "error"
},
"initializationOptions": {
  "checkOnSave": {
    "command": "clippy"
  },
  "cargo": {
    "allFeatures": true,
    "loadOutDirsFromCheck": true
  },
  "procMacro": {
    "enable": true
  }
}
}
}
}
}
}
}
}

```

User prompt: where Cargo.toml lives in ROS2 Rust/Python system, in each package?

Response: **Yes, every Rust package must have its own Cargo.toml at its package root, sitting directly alongside package.xml.** Python packages do not use Cargo.toml unless they compile native Rust extensions (e.g., via PyO3).

To get modern IDE support (rust-analyzer, cargo check, workspace lints) while keeping colcon happy, the standard architecture uses a **two-tier layout**:

Recommended Layout

```

my_ros2_ws/
├── Cargo.toml                <-- Virtual Cargo workspace (for rust-analyzer & linting)
├── src/
│   ├── my_rust_bridge/      <-- Rust ROS 2 Package
│   │   ├── Cargo.toml      <-- Per-package Cargo manifest (for ament_cargo / colcon)
│   │   ├── package.xml     <-- build_type: ament_cargo
│   │   └── src/
│   │       └── main.rs
│   ├── my_python_teleop/    <-- Pure Python ROS 2 Package (No Cargo.toml)
│   │   ├── package.xml     <-- build_type: ament_python
│   │   ├── pyproject.toml  <-- Modern Python build packaging
│   │   └── my_python_teleop/
│   │       └── node.py
│   └── my_robot_sim/        <-- Gazebo / URDF description package

```

```
├── package.xml      <-- build_type: ament_cmake
├── CMakeLists.txt
└── urdf/
```

How the Two Tiers Work

1. Per-Package Cargo.toml (Mandatory for colcon)

Located at `src/<rust_pkg_name>/Cargo.toml`.

colcon discovers ROS 2 packages by scanning for `package.xml`. When it sees `<build_type>ament_cargo</build_type>`, it invokes Cargo directly inside that specific package subdirectory. That directory **must** have its own `Cargo.toml` with the crate's `[package]` declaration:

```
# src/my_rust_bridge/Cargo.toml
[package]
name = "my_rust_bridge"
version = "0.1.0"
edition = "2021"

[dependencies]
rclrs = "*"
sensor_msgs = "*"
std_msgs = "*"
```

2. Root Virtual Cargo.toml (Crucial for Tooling & LSP)

Located at the root of the repository or workspace (`my_ros2_ws/Cargo.toml`).

While colcon builds packages individually, tools like `rust-analyzer`, `cargo-nextest`, and Antigravity hooks need to see all Rust crates as a unified workspace. A root virtual manifest links them together and applies centralized lint rules without interfering with colcon:

```
# my_ros2_ws/Cargo.toml
[workspace]
resolver = "2"
members = [
    "src/my_rust_bridge",
    # Add any other Rust ROS2 packages here
]

[workspace.lints.clippy]
all = { level = "warn", priority = -1 }
unwrap_used = "warn"
```

Python Packages

- **Pure Python nodes:** Do not add a `Cargo.toml`. Use standard modern Python packaging (`pyproject.toml` + `package.xml` with `<build_type>ament_python</build_type>`).

- **Hybrid nodes (Python + Rust via PyO3/Maturin):** These live in a single package containing both `package.xml` and a `Cargo.toml` that configures `crate-type = ["cdylib"]`, built using `colcon-cargo` or custom CMake bindings.
-

User prompt: then create a skill for Cargo rather than Cargo.toml itself

Response: Save this file to `.agents/skills/cargo-toolchain/SKILL.md`. It equips the agent to manage the two-tier workspace, maintain synchronization between Cargo and ROS `package.xml` manifests, and invoke modern Cargo tooling (`nextest`, `clippy`, `machete`, `deny`) correctly within a ROS 2 environment.

```
---
name: "cargo-toolchain"
description: "Orchestrating modern Cargo operations, virtual workspace synchronization, colcon/ament_cargo interop, and quality audits."
triggers:
  - "Cargo.toml"
  - "**/Cargo.toml"
  - "src/**/*rs"
  - "cargo *"
---
```

Cargo Workspace & Toolchain Orchestrator

1. Two-Tier Workspace Topography

The workspace uses a dual-layer manifest strategy to decouple local developer tooling from ``colcon`` build artifacts:

```
* **Root Virtual Workspace (`./Cargo.toml`):**
* Must contain only `[workspace]` configuration (no `[package]`).
* Explicitly lists all Rust leaf packages under `members`.
* Centralizes compiler/clippy lints (`[workspace.lints]`) and shared dependency versions (`[workspace.dependencies]`).
* **Leaf Package Manifests (`src/<rust_pkg>/Cargo.toml`):**
* Defines the concrete crate (`[package]`) and exports.
* Must sit beside `package.xml` in the same directory.
* Inherits shared dependencies and workspace lints via `workspace = true`.
```

Workspace Registration Invariant

Whenever a new Rust package is added to ``src/``:

1. Check ``src/<new_pkg>/package.xml`` has `<build_type>ament_cargo</build_type>``.
2. Append `"src/<new_pkg>"` to ``members`` in the root ``Cargo.toml``.
3. Verify membership: ``cargo metadata --format-version 1 --no-deps``.

2. Manifest Dual-Sync Rule (`Cargo.toml` $\neq$ `package.xml`)

When adding or updating dependencies, both manifests must remain synchronized to prevent ``rosdep`` and CI build failures:

```
* **ROS Interfaces & Client Libraries:** If adding a ROS crate (e.g., `rclrs`, `sensor_msgs`, `geometry_msgs`), ensure a matching <depend>
* **Pure Rust Crates:** Crates with no ROS C-bindings (e.g., `serde`, `glam`, `crossbeam`) only go into `Cargo.toml`.
```

```

```toml
In leaf Cargo.toml:
[dependencies]
rclrs.workspace = true
sensor_msgs.workspace = true
glam = "0.28"
```

```

```

```xml
<!-- In leaf package.xml: -->
<depend>rclrs</depend>
<depend>sensor_msgs</depend>
<!-- Do NOT add glam here -->
```

```

3. Toolchain Execution Playbook

A. Testing via `cargo-nextest`

Use `nextest` for parallel execution and structured isolation. Source the ROS environment before running tests if crates link to `rclrs`:

```
```bash
```

```
Workspace unit & integration tests
```

```
source /opt/ros/$ROS_DISTRO/setup.bash && cargo nextest run --workspace --all-targets
```

```
Isolate a single failing test
```

```
cargo nextest run -E 'test(test_coordinate_inversion)' --no-capture
```

```
```
```

B. Workspace Audits & Dependency Hygiene

* **Dead Dependency Scanning:** Run `cargo machete` to detect unused crates in leaf manifests.

* **Security & License Audits:** Run `cargo deny check` against `deny.toml` to guard against disallowed licenses or vulnerable crates.

* **Linting:** Run `cargo clippy --workspace --all-targets -- -D warnings`.

C. Colcon vs. Cargo Separation of Concerns

Never run `colcon build` for standard compile-edit-test cycles. Distinguish operational boundaries:

| Task | Command | Reason |
|-------------------------|--|---|
| **Syntax / Type Check** | `cargo check --workspace` | Fast (~1-2s), runs via memory caches. |
| **Unit Testing** | `cargo nextest run` | Isolated test runners, readable failure trees. |
| **Full Assembly** | `colcon build --packages-select <pkg>` | Only needed when generating ROS message headers, preparing symlinks in `install`. |

| Task | Command | Reason |
|-------------------------|--|---|
| **Syntax / Type Check** | `cargo check --workspace` | Fast (~1-2s), runs via memory caches. |
| **Unit Testing** | `cargo nextest run` | Isolated test runners, readable failure trees. |
| **Full Assembly** | `colcon build --packages-select <pkg>` | Only needed when generating ROS message headers, preparing symlinks in `install`. |

| Task | Command | Reason |
|-------------------------|--|---|
| **Syntax / Type Check** | `cargo check --workspace` | Fast (~1-2s), runs via memory caches. |
| **Unit Testing** | `cargo nextest run` | Isolated test runners, readable failure trees. |
| **Full Assembly** | `colcon build --packages-select <pkg>` | Only needed when generating ROS message headers, preparing symlinks in `install`. |

| Task | Command | Reason |
|-------------------------|--|---|
| **Syntax / Type Check** | `cargo check --workspace` | Fast (~1-2s), runs via memory caches. |
| **Unit Testing** | `cargo nextest run` | Isolated test runners, readable failure trees. |
| **Full Assembly** | `colcon build --packages-select <pkg>` | Only needed when generating ROS message headers, preparing symlinks in `install`. |

| Task | Command | Reason |
|-------------------------|--|---|
| **Syntax / Type Check** | `cargo check --workspace` | Fast (~1-2s), runs via memory caches. |
| **Unit Testing** | `cargo nextest run` | Isolated test runners, readable failure trees. |
| **Full Assembly** | `colcon build --packages-select <pkg>` | Only needed when generating ROS message headers, preparing symlinks in `install`. |

4. Diagnostics & Common Failure Recovery

* **Linking Error (`librcl.so: cannot open shared object file`):**

* **Cause:** Cargo was executed in a subshell without sourced ROS 2 underlays.

```
* *Fix:* Prefix with `source /opt/ros/${ROS_DISTRO:-humble}/setup.bash && cargo ...`.
* **Proc-Macro Target Mismatch:**
* *Cause:* Discrepancy between host `rustc` and the sysroot targeted by ROS.
* *Fix:* Run `rustup default stable` and clean the local target directory: `cargo clean -p <crate_name>`.
* **Missing C Library Bindings (`bindgen` / `clang` failures):**
* *Cause:* Missing underlying ROS C packages.
* *Fix:* Identify the missing package using `rosdep check --from-paths src --ignore-src` and install with `rosdep install`.
```

User prompt: ok, next code-review and code-simplifier plugings again, you are making choices as experienced seasoned developer. choose best, modern wide adopted alternatives for antigravity.

Response: Senior developers avoid delegating style checks or formatting to an LLM—formatters (rustfmt, prettier) and static linters (clippy, eslint) already handle that deterministically.

Instead, an enterprise-grade setup for **Code Review** and **Code Simplification** isolates review tasks to a detached, read-only sub-agent (preventing context pollution and self-confirmation bias) and pairs refactoring with test-driven invariant verification.

Architecture Overview

| Claude Plugin | Antigravity Native Replacement | Mechanism |
|-----------------|---|--|
| code-review | .agents/agents/code-reviewer.md +
.agents/commands/review.md | Isolated read-only sub-agent. Evaluates git diff for concurrency bugs, coordinate drift, hot-path allocations, and architecture regressions. Structured by severity (P0–P3). |
| code-simplifier | .agents/skills/code-simplifier/SKILL.md +
.agents/commands/simplify.md | Test-driven refactoring skill. Flattens cyclomatic complexity, removes premature abstractions (YAGNI), and verifies test suites pass before and after edits. |

1. Code Review Setup

A. Read-Only Sub-Agent (.agents/agents/code-reviewer.md)

Running reviews inside the primary context window leads to self-confirmation bias (the agent approves its own mistakes). Running it inside a dedicated sub-agent provides an isolated, unbiased perspective without write permissions:

```
---
name: "code-reviewer"
description: "Senior Principal Engineer auditing code changes for correctness, safety, and performance."
tools: ["read_file", "run_command"]
denied_tools: ["write_to_file", "replace_file_content"]
---
```

You are a Senior Principal Systems Engineer auditing changes in a ROS2, Rust, and Three.js workspace.

Audit Focus Areas

- 1. **Concurrency & Thread Safety:** Lock contention, deadlocks, holding locks across async yields, atomics ordering.

2. **Real-Time / Hot-Path Hygiene:** Unnecessary allocations (`Vec`, `String`, `clone()`) in ROS subscription callbacks or Three.js animation.
3. **Robotics Correctness:** REP-103 vs WebGL coordinate conversions, unhandled `Result` / `Option` branching, and URDF limits.
4. **API Ergonomics:** Over-engineering, leaky abstractions, and interface breaking changes.

Output Severity Hierarchy

Organize feedback strictly into:

- * **[P0] Blocker:** Correctness bug, data race, memory leak, or coordinate system inversion.
- * **[P1] Performance:** Unnecessary allocation on hot telemetry loops or blocking operations in callbacks.
- * **[P2] Architectural / Idiomatic:** Non-idiomatic Rust/TypeScript, missed pattern matching, or missing domain error types.
- * **[Nit]:** Minor readability suggestions. Ignore anything already covered by `clippy` or `rustfmt`.

B. Review Slash Command (.agents/commands/review.md)

Invokes the reviewer sub-agent against current Git status:

```
---
description: "Executes an isolated, rigorous code review on uncommitted changes or branch diffs"
---
1. Inspect the workspace status:
   `git diff --staged` (or `git diff main...HEAD` if branch is specified)
2. Switch context to the `code-reviewer` agent.
3. Review the diff strictly against the criteria in `.agents/agents/code-reviewer.md`.
4. Return prioritized findings with concrete, line-level code suggestions.
```

2. Code Simplification Setup

Simplification should never be speculative. It must be gated behind existing unit and integration tests to ensure behavior remains identical while cyclomatic complexity drops.

A. Refactoring Skill (.agents/skills/code-simplifier/SKILL.md)

Save this to .agents/skills/code-simplifier/SKILL.md:

```
---
name: "code-simplifier"
description: "Reduces cognitive complexity, flattens deep branching, and removes speculative abstractions while maintaining test invariants."
triggers:
  - "/simplify"
  - "refactor *"
---
```

Code Simplification Playbook

Principles of Simplification

- * **Eliminate Premature Generalization (YAGNI):** Collapse single-implementation traits, redundant factory functions, and speculative genericity.
- * **Flatten Cyclomatic Complexity:**
 - * Replace nested `if / else` blocks with early returns (guard clauses).
 - * In Rust, replace deeply nested `match` statements with `let-else` statements, combinators (`and\_then`, `map`), or the `?` operator.
 - * In TypeScript, avoid excessive ternaries; use discriminated unions and pattern matching patterns.

```
* **Normalize Data Flow:** Make data move in one direction. Avoid mutable accumulator parameters where an iterator chain (`filter`, `map`, `
* **Preserve Invariants:** Do not alter public function signatures, coordinate conventions, or serialization outputs.
```

Invariant Verification Workflow

Whenever simplifying a file or module:

1. ****Pre-flight Check:**** Run existing tests to establish a baseline:
 - Rust: `cargo nextest run --workspace`
 - Web: `npm --prefix web run test`If tests fail before starting, stop and notify the user.
2. ****Apply Transformation:**** Simplify the target code in small, focused chunks.
3. ****Post-flight Verification:**** Re-run the tests. If anything regresses, revert immediately.
4. ****Linter Validation:**** Ensure changes adhere to workspace lint tables:
 - Rust: `cargo clippy --workspace --all-targets`
 - Web: `npm --prefix web run lint`

B. Simplification Slash Command (.agents/commands/simplify.md)

```
---
description: "Safely simplifies targeted code or functions using test-driven refactoring"
---
```

1. Identify target file or function provided in the arguments.
2. Run baseline verification:
 - `cargo nextest run -p <pkg>` (or `npm --prefix web test`)
3. Apply simplification guidelines from `.agents/skills/code-simplifier/SKILL.md`:
 - Replace nested branching with guard clauses/early returns.
 - Remove unused abstractions, traits, and wrappers.
4. Run verification to guarantee zero behavioral regressions:
 - Run tests again.
 - Run `cargo clippy --message-format=short` (or web linter).
5. Output a summary showing before/after cyclomatic complexity and line counts.

How to Use

- To run an isolated review before committing:

```
/review
```

- To reduce complexity on a tangled ROS2 node or Three.js component:

```
/simplify src/rust_sim_bridge/src/main.rs
```

User prompt: next pr-review-toolkit

Response: In Claude Code, pr-review-toolkit packages multi-faceted GitHub PR workflows: generating PR summaries, checking breaking API changes, performing pre-flight sanity checks, and leaving structured review comments.

For a senior engineering harness in Antigravity, we delegate local git and GitHub operations to the official **GitHub CLI (gh)** rather than heavy MCP layers, paired with automated SemVer verification (**cargo-semver-checks**) and ROS message compatibility checks.

Harness Architecture

| Component | File Path | Functionality |
|-------------------|------------------------------------|---|
| PR Engine Skill | .agents/skills/pr-toolkit/SKILL.md | Pre-flight validation, SemVer breaking change detection, and PR template population. |
| PR Create Command | .agents/commands/pr-create.md | Automates /pr-create: audits branch, runs test gates, crafts a conventional PR body, and opens a PR via gh. |
| PR Review Command | .agents/commands/pr-review.md | Automates /pr-review <pr#>: fetches remote PR diffs, executes test suites, and generates line-anchored review comments. |

1. PR Toolkit Skill (.agents/skills/pr-toolkit/SKILL.md)

Save this to .agents/skills/pr-toolkit/SKILL.md:

```
---
name: "pr-toolkit"
description: "End-to-end pull request lifecycle: pre-flight checks, SemVer validation, changelog drafting, and code review."
triggers:
  - "/pr-*"
  - "gh pr *"
---

# Pull Request Toolkit Playbook

## 1. Tooling Prerequisites
* **GitHub CLI (`gh`):** Use `gh` for all GitHub interactions (`gh pr create`, `gh pr diff`, `gh pr comment`).
* **Rust SemVer Guard:** Use `cargo-semver-checks` to verify public API stability if crate versions are unchanged.

## 2. Pre-Flight Gate (Must Pass Before PR Creation)
Never open or approve a PR if any step fails:
1. **Dirty Tree Check:** `git status --porcelain` must be clean.
2. **Rust Test & Lint Pass:**
  ```bash
 cargo clippy --workspace --all-targets -- -D warnings
 cargo nextest run --workspace --all-targets
  ```
3. **Web Frontend Pass:**
  ```bash
 npm --prefix web run typecheck
 npm --prefix web run test
  ```
```

4. ****ROS2 Interface Breaking Change Check:****

Verify if any `.msg`, `.srv`, or `.action` files were modified. Modifying existing field IDs or types without a major version bump is for

3. Pull Request Body Standard

Every PR generated by the agent must adhere to this Markdown structure:

```markdown

#### ## Summary

- Concise bullet points summarizing the core changes and why they were made.

#### ## Architectural Changes & Invariants

- **\*\*ROS2/Gazebo:\*\*** (Changes to topics, QoS, URDF, frames, or physics parameters)
- **\*\*Rust/Web:\*\*** (Performance implications, API updates, or coordinate sync changes)

#### ## Breaking Changes & SemVer

- [ ] Yes (Describe breaking API changes or message definition modifications)
- [ ] No (Backward-compatible change)

#### ## Validation Performed

- [x] Unit/Integration tests: ``cargo nextest run`` passed
- [x] Static analysis: ``cargo clippy`` and ``npm run typecheck`` passed
- [x] Visual validation: Simulation verified via Three.js visualizer

### ## 4. Review Evaluation Criteria (For Reviewing External PRs)

When auditing incoming PRs:

- \* **\*\*Blast Radius:\*\*** Trace every modified ROS topic and verify subscriber/publisher count.
- \* **\*\*QoS Alignment:\*\*** Confirm publisher and subscriber QoS policies match (``Reliable`` vs ``BestEffort``, transient local vs volatile).
- \* **\*\*Memory Invariants:\*\*** Check that no high-frequency loops allocate on the heap or leak listeners.

## 2. PR Creation Command (`.agents/commands/pr-create.md`)

Save this to `.agents/commands/pr-create.md`. It executes all test suites, checks for uncommitted files, generates a standardized PR description, and pushes the branch:

```

description: "Runs quality gates, generates structured summary, and opens a GitHub PR"

```

1. Check current branch:  
Ensure current branch is not ``main`` or ``master``.
2. Run pre-flight verification:
  - ``cargo clippy --workspace --all-targets -- -D warnings``
  - ``cargo nextest run --workspace``
  - ``npm --prefix web run typecheck``Stop execution immediately if any check fails.
3. Check for API breaking changes:  
Run ``cargo semver-checks check-release`` if package versions are not bumped.
4. Push current branch to remote:  
``git push -u origin HEAD``

5. Generate PR description using the template from ``.agents/skills/pr-toolkit/SKILL.md``.
6. Open the PR via GitHub CLI:  
``gh pr create --title "<type>(<scope>): <summary>" --body "<generated_body>"``
7. Return the resulting PR URL.

### 3. PR Review Command (`.agents/commands/pr-review.md`)

Save this to `.agents/commands/pr-review.md`. It downloads the target PR's diff, analyzes it in a read-only pass, and presents prioritized feedback:

```

description: "Fetches and audits a remote PR using isolated code-reviewer rules"

1. Identify PR number from arguments (e.g. `/pr-review 42`).
2. Fetch PR metadata and diff:
 `gh pr diff <PR_NUMBER>`
3. Fetch PR discussion and comments:
 `gh pr view <PR_NUMBER> --comments`
4. Evaluate changes against `.agents/skills/pr-toolkit/SKILL.md` and `.agents/agents/code-reviewer.md`:
 - Inspect thread-safety and lock-free channels.
 - Verify REP-103 coordinate transforms.
 - Verify ROS2 message QoS compatibility.
5. Provide review output formatted by severity ([P0] Blocker, [P1] Performance, [P2] Suggestion, [Nit]).
6. (Optional) If instructed with `--submit`, post the review using:
 `gh pr review <PR_NUMBER> --comment -b "<review_body>"`
```

### How to Use

- **Open a verified PR from your current branch:**

```
/pr-create
```

- **Audit an open PR from a teammate:**

```
/pr-review 12
```

---

*User prompt: next, i like this plugin and absolutely should have it in antigravity I am talking about feature-dev*

Response: feature-dev provides a disciplined, multi-phase feature development workflow. Instead of jumping straight into writing code, it guides the process through four distinct phases: **requirements gathering & scope grilling**, **codebase exploration**, **architectural design / RFC**, and **phased test-driven implementation**.

In AntigraVity CLI, you implement this with a dedicated skill (`.agents/skills/feature-dev/SKILL.md`) and a slash command (`.agents/commands/feature-dev.md`).

### 1. Feature Development Skill (`.agents/skills/feature-dev/SKILL.md`)

Save this to `.agents/skills/feature-dev/SKILL.md`:

```

name: "feature-dev"
description: "Structured multi-phase feature engineering: discovery, architectural RFC, test-first planning, and execution."
triggers:
 - "/feature*"
 - "implement feature *"

```

## # Feature Development Protocol

Never write production code without completing Phases 1 through 3. Every feature must follow this four-phase lifecycle:

```
```text
Discovery & Grilling → Architecture & RFC → Verification Plan → Phased Execution
```
```

---

### ## Phase 1: Discovery & Technical Grilling

Before proposing solutions, aggressively scope requirements:

1. **Domain Boundary Check:**
  - Does this require a new ROS2 topic, service, or action?
  - What are the QoS requirements (`Transient Local` vs. `Volatile`, `Reliable` vs. `Best Effort`)?
  - Does this touch Three.js rendering loops or change REP-103 coordinate conventions?
2. **Clarifying Questions:** Ask up to 3 targeted questions uncovering edge cases, latency budgets, or fallback behavior. Do not proceed until

---

### ## Phase 2: Codebase Exploration & Impact Analysis

1. Map affected files using `graphify query` or `rust-analyzer` references.
2. Identify existing traits, types, or messages that can be reused rather than duplicated.
3. Check for performance implications:
  - Does this introduce allocations on hot telemetry paths?
  - Will this block the `rclrs` executor or Three.js `requestAnimationFrame`?

---

### ## Phase 3: Architectural RFC & Spec

Produce a compact RFC markdown block covering:

- \* **Interface Contract:** Exact Rust struct/enum definitions, ROS2 `.msg` schemas, or TypeScript interfaces.
- \* **Data Flow:** ASCII diagram or step-by-step pipeline from publisher/Gazebo to visualizer.
- \* **Invariants:** What assumptions must hold true (e.g., "joint angles must remain bounded between lower and upper URDF limits").

---

### ## Phase 4: Test-First Task Matrix

Break implementation into small, atomic commits. Each task must specify its test gate:

```

* [] **Task 1: Interfaces & Stubs**
 - Define message types, Rust traits, or TypeScript models.
 - *Gate:* `cargo check --workspace`
* [] **Task 2: Failing Test Harness (TDD)**
 - Write unit tests demonstrating the desired behavior or coordinate conversion.
 - *Gate:* `cargo nextest run -p <pkg>` fails with expected error.
* [] **Task 3: Implementation**
 - Implement business logic and wire up callbacks/subscribers.
 - *Gate:* Tests pass, `cargo clippy` and `npm run typecheck` pass.
* [] **Task 4: Simulation Smoke Test**
 - Verify message flow or rendering behavior with the simulation running.

```

## 2. Feature Development Command (.agents/commands/feature-dev.md)

Save this to .agents/commands/feature-dev.md. This command binds user input to the feature lifecycle, ensuring the agent pauses for your approval before writing production code:

```

description: "Executes a structured feature development cycle (Grill -> Spec -> Plan -> Execute)"

1. Parse feature objective from command arguments:
 `$ARGUMENTS`
2. **Phase 1: Grilling**
 Analyze the feature against existing packages in `src/` and `web/`.
 If any ambiguity exists regarding ROS QoS, coordinate frames, or state synchronization, ask focused questions and wait for user response.
3. **Phase 2: Discovery**
 Inspect relevant AST structures, types, and existing tests using LSP or file inspection.
4. **Phase 3: RFC & Plan Formulation**
 Formulate the architectural design and Phase 4 Task Matrix as specified in `.agents/skills/feature-dev/SKILL.md`.
5. **Approval Gate:**
 Present the RFC and Task Matrix to the user.
 STOP AND WAIT FOR USER CONFIRMATION. Do not proceed to implementation until the user approves or adjusts the design.
6. **Phase 4: Phased Execution**
 Execute tasks sequentially, validating each task against its compiler and test gates (`cargo nextest`, `typecheck`).

```

## 3. Usage Workflow

Initiate a new feature run:

/feature-dev Add a real-time IMU orientation visualizer using Three.js and an rclrs subscriber

The agent will execute as follows:

1. **Questions:** Asks about message frequency (e.g., 100 Hz vs 10 Hz), coordinate transforms (REP-103 to Three.js quaternion), and whether to display raw orientation or an oriented 3D arrow gizmo.
2. **RFC:** Outlines the sensor\_msgs/msg/Imu listener in Rust, the WebSocket payload contract, and the Preact quaternion hook.
3. **Approval Gate:** Halts to let you inspect and tweak the proposed architecture.

4. **TDD Execution:** Implements the unit tests, then the logic, validating each commit with cargo-nextest and typecheck.

---

User prompt: and last but not least *skill-creator*

Response: The *skill-creator* tool functions as a meta-harness capability: it extracts recurring workflows, debugging recipes, or domain-specific protocols into standardized, self-contained Agent Skills.

In Antigravity CLI, this translates into a meta-skill (`.agents/skills/skill-creator/SKILL.md`) that defines the generation contract, and an executable slash command (`.agents/commands/create-skill.md`) that scaffolds, validates, and indexes new skills on demand.

### 1. Skill Creator Meta-Skill (`.agents/skills/skill-creator/SKILL.md`)

Save this to `.agents/skills/skill-creator/SKILL.md`. It teaches the agent the exact anatomy, trigger hygiene, and quality standards required for new skills in your ROS2/Rust/Three.js workspace:

```

name: "skill-creator"
description: "Authoring, structuring, and optimizing new reusable Agent Skills adhering to the Agent Skills standard."
triggers:
 - "/create-skill *"
 - "create skill *"
 - "author skill *"

Agent Skill Generation Standard

Core Philosophy
A skill is not a general tutorial or documentation dump. A skill is an executable operational protocol for an AI agent. It must contain:
1. Targeted triggers: File globs or keywords that strictly scope when the skill enters the agent's context window.
2. Invariants & Non-negotiables: Constraints the agent must never violate (e.g., coordinate frames, memory safety, QoS matches).
3. Concrete code templates: Idiomatic snippets ready for immediate adaptation.
4. Failure modes & recovery: Exact error messages and the mechanical steps to resolve them.

Skill Directory Layout
Every generated skill must live in its own directory under .agents/skills/`:

```text
.agents/skills/<skill-name>/
├── SKILL.md           # Primary operational playbook
├── references/        # (Optional) Complex schemas, templates, or scripts
└── template.rs
```

```

## ## Frontmatter Schema Requirements

The generated `SKILL.md` must begin with valid YAML frontmatter:

```
```yaml
---
name: "kebab-case-name"
description: "High-density summary of capability and domain boundaries."
triggers:
  - "path/glob/**/*.{ext}"
  - "slash-command"
  - "domain-keyword"
---
```

Trigger Hygiene

- * **Avoid universal wildcards:** Never use `\*` or generic triggers like `code` or `dev`.
- * **Path-based triggering:** Bind to relevant file types (e.g., `src/\*\*/\*.rs`, `sim/\*\*/\*.sdf`, `web/\*\*/\*.tsx`).
- * **Semantic keywords:** Target specific tooling or frameworks (e.g., `gz topic`, `rclrs:spin`, `urdf-loader`).

Body Composition Template

Structure every generated `SKILL.md` using this exact hierarchy:

1. **Title & Purpose:** Single-sentence operational boundary.
2. **Architecture / Pipeline:** Minimal ASCII diagram or data flow path.
3. **Canonical Implementation Pattern:** A concrete, copy-pasteable, verified code example using workspace defaults.
4. **Hard Invariants:** Bulleted list of safety or architectural rules.
5. **Common Failure Modes:** A table or bulleted list pairing symptoms with mechanical fixes.

2. Skill Generation Command (.agents/commands/create-skill.md)

Save this to .agents/commands/create-skill.md. This gives you a /create-skill command to scaffold new skills interactively or distill recent troubleshooting sessions:

```
---
description: "Scaffold a new skill or distill the current terminal session into a reusable skill"
---
1. Parse skill objective from command arguments:
  ` $ARGUMENTS `

2. **Extraction Mode Evaluation:**
  - If ` $ARGUMENTS ` specifies a new topic (e.g., `/create-skill gazebo-sdf-sensor`): Scrape domain knowledge, existing repository patterns,
  - If ` $ARGUMENTS ` is `from-session`: Analyze the recent conversation turns, terminal outputs, and bug fixes to identify the solved problem

3. **Scaffolding:**
  - Generate the target directory: `.agents/skills/<skill-name>/`
  - Create `.agents/skills/<skill-name>/SKILL.md` strictly following the schema in `.agents/skills/skill-creator/SKILL.md`.
```

4. ****Frontmatter Verification:****
 - Confirm `name` is valid kebab-case.
 - Confirm `description` is informative and concise.
 - Confirm `triggers` are scoped strictly (no overly broad globs).
5. ****Registration & Indexing:****
 - If `.agents/settings.json` enforces an allowlist under `skills.enabled`, append `` to that list.
 - Output the file path and a 3-bullet summary of the newly created skill.

Usage Examples

1. Create a skill for a specific robotics task:

```
/create-skill ros2-gazebo-clock-bridge
```

The agent inspects your ROS2 workspace, builds `.agents/skills/ros2-gazebo-clock-bridge/SKILL.md`, configures triggers for `sim/**/*.launch.py` and `sim/worlds/*.sdf`, and provides launch patterns for syncing simulation time (`/clock`) with Rust nodes.

2. Distill a hard-won debugging session into permanent memory:

```
/create-skill from-session
```

If you spent 10 turns debugging an elusive Three.js WebGL context loss or a Gazebo ignition sensor plugin linking failure, running this extracts the exact symptoms, root cause, and reproducible fix directly into a permanent skill.
